

Master Java Career Curriculum for MCA Freshers

Prepared by: Senior Java Career Advisor / Java Architect / Technical Hiring Expert

Target Audience: MCA fresher with basic programming knowledge, no professional experience, aiming for first entry-level Java job.

Guiding Principle: Learn common foundations once, reuse them across all specializations. Do not chase every technology. Focus on what gets you hired.

PART 1 — JAVA COMMON FOUNDATION

These are the skills genuinely shared across **all** Java career paths. If a topic is not listed here, it belongs to a specialization.

Dependency Order (Learn in This Sequence)

1. Programming Fundamentals
2. Java Syntax & Basics
3. OOP
4. Core Java APIs (Exceptions, Collections, Generics, I/O, Date/Time)
5. Java 8+ Features (Lambdas, Streams, Optional)
6. Multithreading & Concurrency Fundamentals
7. JVM Fundamentals + Memory + GC
8. Tools: Git, Maven/Gradle, Linux, Debugging, Unit Testing
9. HTTP & Basic Networking
10. Software Engineering Practices

Chapter 1: Programming Fundamentals & Java Basics

Field	Details
What to Learn	How programs execute, variables, data types, operators, control flow, methods, arrays, strings
Important Subtopics	Primitive vs reference types, type casting, <code>==</code> vs <code>.equals()</code> , <code>String</code> immutability, <code>StringBuilder</code> vs <code>StringBuffer</code> , enhanced for-loop, varargs

Prerequisites	None
Fresher Depth	Write correct, clean Java code for simple problems. Know syntax cold.
Practical Exercises	Calculator, number guessing game, palindrome checker, array sorting without library, string reversal, matrix operations
Common Interview Topics	<code>==</code> vs <code>equals()</code> , <code>String</code> pool, pass-by-value semantics, <code>final</code> keyword, operator precedence

Chapter 2: Object-Oriented Programming (OOP)

Field	Details
What to Learn	Classes, objects, inheritance, polymorphism, abstraction, encapsulation, interfaces, abstract classes
Important Subtopics	<code>this</code> vs <code>super</code> , method overloading vs overriding, covariant return types, interface default/static methods, abstract class vs interface, composition vs inheritance, <code>Object</code> class methods (<code>toString</code> , <code>hashCode</code> , <code>equals</code>)
Prerequisites	Chapter 1
Fresher Depth	Design small class hierarchies. Explain all four pillars with real examples.
Practical Exercises	Library management (books, members, loans), shape area calculator (polymorphism), employee hierarchy, bank account system
Common Interview Topics	Overloading vs overriding, abstract class vs interface, why multiple inheritance is not supported, <code>hashCode</code> / <code>equals</code> contract, SOLID principles (basic)

Chapter 3: Core Java APIs

Field	Details
What to Learn	Exception handling, Collections Framework, Generics, Java I/O, Date and Time API
Important Subtopics	Checked vs unchecked exceptions, try-with-resources, custom exceptions; <code>List</code> , <code>Set</code> , <code>Map</code> , <code>Queue</code> , <code>ArrayList</code> vs <code>LinkedList</code> , <code>HashMap</code> vs <code>TreeMap</code> , <code>HashSet</code> vs <code>TreeSet</code> ; generic classes/methods, bounded type parameters, wildcards; file reading/writing, serialization basics; <code>LocalDate</code> , <code>LocalDateTime</code> , <code>ZonedDateTime</code> , <code>Duration</code> , <code>Period</code>
Prerequisites	Chapters 1–2
Fresher Depth	Use collections fluently. Handle exceptions properly. Read/write files. Use modern Date/Time.
Practical Exercises	Word frequency counter (Map), duplicate remover (Set), student grade tracker (List + sorting), file-based todo list, generic pair class
Common Interview Topics	<code>HashMap</code> internal working, <code>ArrayList</code> vs <code>LinkedList</code> , fail-fast vs fail-safe iterators, <code>Comparable</code> vs <code>Comparator</code> , checked vs unchecked exceptions, try-with-resources

Chapter 4: Java 8+ Features

Field	Details
What to Learn	Lambda expressions, Stream API, Optional, functional interfaces
Important Subtopics	Predicate , Function , Consumer , Supplier , method references, Stream operations (filter, map, reduce, collect, flatMap), Collectors , parallel streams (awareness), Optional usage and anti-patterns
Prerequisites	Chapters 2–3
Fresher Depth	Write streams for real data processing. Use lambdas comfortably.
Practical Exercises	Employee salary analysis, order processing pipeline, log file parser with streams, filter/sort/group collections
Common Interview Topics	Lambda vs anonymous class, intermediate vs terminal operations, map vs flatMap , Optional best practices, functional interface definition

Chapter 5: Multithreading & Concurrency Fundamentals

Field	Details
What to Learn	Threads, synchronization, concurrent collections, basic ExecutorService
Important Subtopics	Thread lifecycle, Runnable vs Thread , synchronized , volatile , AtomicInteger , ConcurrentHashMap , ExecutorService , Future , deadlock basics
Prerequisites	Chapters 1–4
Fresher Depth	Understand concepts. Write simple multithreaded programs. Know why concurrency matters.
Practical Exercises	Producer-consumer (BlockingQueue), thread-safe counter, parallel file downloader simulation
Common Interview Topics	synchronized vs Lock , volatile vs Atomic , thread pool basics, deadlock conditions, ConcurrentHashMap vs HashMap

Note: Deep concurrency is a specialization topic. Freshers need conceptual clarity + basic usage.

Chapter 6: JVM, Memory Management & Garbage Collection

Field	Details
What to Learn	JDK vs JRE vs JVM, class loading, memory areas, garbage collection basics
Important Subtopics	Heap vs stack, metaspace, young/old generation, GC roots, minor vs major GC, System.gc() (why not to use), basic JVM flags
Prerequisites	Chapters 1–5
Fresher Depth	Explain JVM architecture. Know what causes OutOfMemoryError .
Practical Exercises	Monitor a simple app with jconsole or jvisualvm , trigger and observe GC

Common Interview Topics	JVM architecture, heap structure, GC types, classloader delegation model, <code>StackOverflowError</code> vs <code>OutOfMemoryError</code>
--------------------------------	--

Chapter 7: Tools & Engineering Practices

Field	Details
What to Learn	Git/GitHub, Maven/Gradle, Linux/command line, debugging, unit testing, HTTP basics, software engineering practices
Important Subtopics	Git: branch, merge, rebase, PR workflow; Maven: <code>pom.xml</code> , dependencies, lifecycle, plugins; Linux: file navigation, permissions, processes, <code>grep</code> , <code>ssh</code> ; Debugging: breakpoints, step-through, watch; Testing: JUnit 5, assertions, <code>@BeforeEach</code> , <code>@AfterEach</code> , mocking basics (Mockito); HTTP: methods, status codes, headers, statelessness; Practices: clean code, code reviews, logging (SLF4J basics), basic design principles
Prerequisites	Chapters 1–6
Fresher Depth	Use Git daily. Build projects with Maven. Navigate Linux. Write unit tests. Understand HTTP request/response.
Practical Exercises	Host a project on GitHub with meaningful commits, write JUnit tests for a calculator, create a Maven project from scratch, use <code>curl</code> to test a public API
Common Interview Topics	Git merge vs rebase, Maven lifecycle, JUnit annotations, HTTP status codes, GET vs POST, idempotency

PART 2 — JAVA CAREER PATHS

Reality Check: How the Industry Actually Groups These

Many “career paths” are overlapping roles. Here is the honest grouping:

Group	Included Roles	Why Grouped
A. Java Backend Developer	Java Developer, Java Backend Developer, Spring Boot Developer, Enterprise Java Developer, Java Software Engineer	Same core job: build server-side applications with Java + Spring. Titles vary by company.
B. Java Full-Stack Developer	Java Full-Stack Developer, Java + Frontend Developer	Backend (Group A) + frontend framework.
C. Java Microservices Developer	Microservices Developer, Distributed Systems Developer	Advanced backend specialization. Not an entry-level role.
D. Java + Cloud Developer	Cloud Developer, Java Cloud Engineer	Backend + cloud platform skills.
E. Java QA/Automation Engineer	SDET, Automation Engineer, QA Engineer (Java)	Testing-focused. Different day-to-day work.

F. Java DevOps-oriented Developer	DevOps Engineer (Java background), Platform Engineer	Operations + development overlap.
G. Java Android Developer	Android Developer	Declining as a Java path. Android is now Kotlin-first. Java is legacy. Not recommended for freshers targeting Java careers.

Entry-Level Suitability

Path	Entry-Level Suitable?	Notes
A. Backend	Yes	Primary target for MCA freshers
B. Full-Stack	Yes	Slightly broader, more competition
C. Microservices	Not really	Needs backend experience first
D. Cloud	Partially	Cloud fundamentals can be learned, but cloud roles often want experience
E. QA/Automation	Yes	Good entry point, different career trajectory
F. DevOps	Partially	Some entry roles exist, but most want ops experience
G. Android	Avoid	Kotlin-first now

Detailed Path Breakdown

A. Java Backend Developer

Field	Details
What they do	Build and maintain server-side logic, APIs, database interactions, business rules
Common Titles	Java Developer, Backend Developer, Software Engineer (Java), Spring Boot Developer
Common Foundation	All of Part 1
Specialized Skills	Spring Boot, REST, JPA/Hibernate, SQL, Spring Security, Docker basics
Responsibilities	Write APIs, fix bugs, write tests, participate in code reviews, deploy to dev/staging
Entry-Level Suitability	Excellent
Main Technologies	Java 17+, Spring Boot 3.x, PostgreSQL/MySQL, Maven, Git, Docker
Typical Projects	REST API for e-commerce, employee management system, blog backend
Interview Focus	Core Java, Spring Boot, REST, SQL, DSA (easy-medium)

B. Java Full-Stack Developer

Field	Details
What they do	Backend + frontend development

Common Titles	Full-Stack Java Developer, Java + React Developer
Common Foundation	All of Part 1
Specialized Skills	Backend (Spring Boot, JPA, SQL) + Frontend (HTML, CSS, JS, React/Angular)
Entry-Level Suitability	Good, but more to learn
Main Technologies	Spring Boot + React (most common) or Angular
Interview Focus	Backend + frontend basics, how they connect

C. Java Microservices Developer

Field	Details
What they do	Design and build distributed systems as independent services
Entry-Level Suitability	Not entry-level. Learn after 1–2 years of backend experience.
Prerequisites	Strong backend + Spring Boot + Docker + networking

D. Java + Cloud Developer

Field	Details
What they do	Build Java apps deployed on cloud platforms
Entry-Level Suitability	Cloud fundamentals are entry-friendly, but “Cloud Developer” titles often want experience
Main Technologies	AWS/Azure, Docker, Kubernetes, CI/CD

E. Java QA/Automation Engineer

Field	Details
What they do	Write automated tests for web/API applications
Entry-Level Suitability	Good alternative entry point
Main Technologies	Selenium, REST Assured, TestNG/JUnit, Maven, Jenkins
Difference from Developer	Less feature-building, more test framework building and quality ownership

F. Java DevOps-oriented Developer

Field	Details
What they do	Bridge development and operations: CI/CD, containers, deployment
Entry-Level Suitability	Some entry roles, but most want ops experience
Main Technologies	Docker, Kubernetes, Jenkins/GitHub Actions, Linux, Cloud

G. Java Android Developer

Field	Details
-------	---------

Status	Not recommended as a Java-focused path. Android is Kotlin-first since 2019. Java is legacy.
If you insist	Learn Kotlin instead. Java Android jobs are declining.

PART 3 — JAVA BACKEND DEVELOPER ROADMAP

Common Foundation (from Part 1)

All of Part 1 is required.

Backend-Specific Skills (in order)

Order	Skill	Why	Fresher Depth
1	HTTP & REST	Foundation of backend APIs	Understand methods, status codes, headers, statelessness
2	JSON	Data exchange format	Parse, serialize, understand structure
3	SQL	Database interaction	Joins, GROUP BY, subqueries, indexes
4	JDBC	Understand raw DB access	Know what Spring Data JPA abstracts
5	Servlets (basics)	Understand what Spring MVC builds on	Conceptual: request/response lifecycle, filters
6	Spring Core	DI, IoC	Understand beans, injection, configuration
7	Spring Boot	Rapid application development	Build REST APIs, use starters, configuration
8	Spring MVC	Web layer	Controllers, <code>@RequestMapping</code> , <code>@RequestBody</code> , <code>@PathVariable</code>
9	Spring Data JPA + Hibernate	Database layer	Entities, repositories, relationships, queries
10	Validation	Input correctness	<code>@Valid</code> , <code>@NotNull</code> , custom validators
11	Exception Handling	Robust APIs	<code>@ControllerAdvice</code> , custom exceptions, error responses
12	Spring Security + JWT	Auth	Basic auth flow, JWT creation/validation, role-based access
13	API Documentation	Usability	Swagger/OpenAPI annotations
14	Testing	Quality	JUnit 5, Mockito, <code>@SpringBootTest</code> , <code>@WebMvcTest</code>
15	Docker (basics)	Deployment	Write Dockerfile, run container, docker-compose basics
16	Logging	Observability	SLF4J, Logback, log levels

17	Deployment	Real-world	Deploy to a cloud VM or free tier (Render, Railway, AWS EC2)
----	------------	------------	--

What is NOT Needed for Entry-Level Backend

- Kubernetes (learn after joining)
- Kafka/RabbitMQ (unless job asks)
- Microservices patterns
- Advanced cloud architecture
- Reactive programming (WebFlux) — nice to know, not required
- GraphQL — not required

Backend Projects (see Part 15)

PART 4 — JAVA FULL-STACK DEVELOPER ROADMAP

Structure

Common Java Foundation (Part 1)

Backend Specialization (Part 3)

Frontend Specialization

Frontend Specialization (in order)

Order	Skill	Fresher Depth
1	HTML	Semantic tags, forms, basic structure
2	CSS	Flexbox, Grid, responsive basics, one framework (Tailwind or Bootstrap)
3	JavaScript	ES6+, DOM manipulation, fetch API, async/await, promises
4	TypeScript	Types, interfaces, generics (basic)
5	React (or Angular)	Components, props, state, hooks, routing, API integration

How Frontend Connects to Backend

Frontend Concept	Backend Connection
fetch / axios	Calls REST API endpoints
JWT storage	Sent in Authorization: Bearer <token> header
CORS	Configured in Spring Boot
Form validation	Both client-side (JS) and server-side (Spring Validation)
Environment variables	API base URLs per environment

How Much Frontend Does a Java Full-Stack Fresher Need?

Enough to build a complete UI that consumes your own API. You do not need to be a frontend expert. You need:

- Build forms, tables, navigation
- Call APIs and display data
- Handle loading/error states
- Basic styling (not designer-level)
- Understand component lifecycle

Not required initially: Advanced state management (Redux), SSR (Next.js), complex animations, webpack configuration.

PART 5 — SPRING BOOT DEVELOPER ROADMAP

Must-Have for Fresher

Skill	Details
Spring Core	DI, IoC, bean lifecycle, <code>@Component</code> , <code>@Service</code> , <code>@Repository</code> , <code>@Configuration</code>
Spring Boot	Auto-configuration, starters, <code>application.properties</code> /YAML, profiles
Spring MVC	<code>@RestController</code> , <code>@RequestMapping</code> , request/response handling
REST APIs	Design, versioning, status codes, error handling
Spring Data JPA	Repositories, entity mapping, relationships, JPQL, native queries
Hibernate	ORM concepts, caching basics, lazy vs eager loading
Validation	<code>@Valid</code> , <code>@NotNull</code> , <code>@Size</code> , custom validators
Exception Handling	<code>@ControllerAdvice</code> , <code>@ExceptionHandler</code> , standardized error responses
Spring Security	Authentication, authorization, password encoding, basic JWT
Configuration	Profiles, externalized config, <code>@Value</code> , <code>@ConfigurationProperties</code>
Testing	<code>@SpringBootTest</code> , <code>@WebMvcTest</code> , <code>@DataJpaTest</code> , Mockito
Docker	Dockerfile for Spring Boot app, docker-compose with DB
Actuator	Health checks, metrics, info endpoints

Advanced Spring Boot (Learn After Joining)

Skill	Why Later
Spring Cloud	Microservices territory
Spring Batch	Batch processing — niche
Spring Integration	Enterprise integration — niche
Spring WebFlux	Reactive — only if job requires

Advanced Security (OAuth2, SSO)	Complex, job-specific
Custom auto-configuration	Framework-level
Performance tuning	Experience-based

PART 6 — JAVA MICROSERVICES ROADMAP

Prerequisites

Do not attempt microservices before:

- Strong Spring Boot backend skills
- Comfortable with REST, SQL, JPA
- Basic Docker
- Understanding of HTTP and networking
- At least 6–12 months of professional backend experience (or exceptionally strong projects)

Microservices Curriculum

Order	Topic	Fresher Appropriate?
1	Monolithic vs Microservices architecture	Conceptual — yes
2	Domain-driven design basics	Awareness only
3	REST communication between services	Yes
4	Service discovery (Eureka, Consul)	Awareness
5	API Gateway (Spring Cloud Gateway)	Awareness
6	Configuration management (Spring Cloud Config)	Awareness
7	Inter-service communication (Feign, RestTemplate)	Basic usage
8	Resilience (Resilience4j, circuit breakers)	Awareness
9	Distributed transactions (Saga pattern)	Advanced
10	Messaging (Kafka, RabbitMQ)	Awareness, basic usage
11	Docker	Yes
12	Kubernetes fundamentals	Awareness
13	Observability (logging, monitoring, tracing)	Awareness
14	Distributed tracing (Zipkin, Jaeger)	Advanced
15	Cloud deployment	Basic

Fresher Strategy for Microservices

Learn conceptually. Build one small project with 2–3 services. Do not claim microservices expertise. Interviewers will test depth.

PART 7 — JAVA + CLOUD DEVELOPER ROADMAP

Prioritized Cloud Knowledge for Java Developers

Priority	Topic	Why
1	Cloud fundamentals (IaaS, PaaS, SaaS)	Conceptual foundation
2	One cloud provider (AWS recommended)	Market share
3	Compute (EC2, Lambda basics)	Where Java runs
4	Storage (S3)	File storage
5	Databases (RDS, DynamoDB basics)	Managed databases
6	IAM	Security essentials
7	Networking (VPC basics, security groups)	Deployment understanding
8	Docker	Containerization
9	Kubernetes fundamentals	Orchestration awareness
10	CI/CD (GitHub Actions, AWS CodePipeline)	Deployment automation
11	Deploying Spring Boot apps	Practical skill
12	Monitoring (CloudWatch basics)	Operations awareness

What Freshers Should Actually Learn

- Cloud concepts + one provider fundamentals
- Deploy a Spring Boot app to a free tier (AWS EC2, Render, Railway)
- Docker basics
- CI/CD with GitHub Actions

Do not: Get certified before you can deploy a simple app. Certifications without practical skills are not helpful for entry-level.

PART 8 — JAVA QA/AUTOMATION ROADMAP

Java Fundamentals Required

Same as Part 1, but depth can be slightly less on concurrency and JVM. Strong OOP, collections, exceptions, and Java 8+ are still required.

Testing-Specific Skills

Order	Skill	Fresher Depth
1	Testing fundamentals	Test types, test pyramid, test cases

2	JUnit 5	Assertions, lifecycle, parameterized tests
3	TestNG	Alternative framework, suites, groups
4	Selenium WebDriver	Locators, waits, page object model
5	API Testing	REST Assured, Postman
6	Automation Frameworks	Design a simple framework (POM, data-driven)
7	Maven	Build and dependency management
8	Git	Version control
9	CI/CD	Jenkins/GitHub Actions for test execution
10	Test Reporting	ExtentReports, Allure basics

Java Developer vs Java QA/Automation Engineer

Aspect	Developer	QA/Automation
Primary Focus	Build features	Verify features
Code Purpose	Production code	Test code
Design Thinking	Architecture, scalability	Test coverage, reliability
Career Growth	Senior Dev Architect	Senior QA QA Architect / SDET Lead
DSA Requirement	Moderate	Lower
SQL Requirement	Moderate	Moderate
Spring Requirement	High	Low-Moderate

PART 9 — JAVA DEVOPS-ORIENTED DEVELOPER ROADMAP

What a Java Fresher Should Learn

Skill	Priority	Why
Linux basics	Must	Everything runs on Linux
Git	Must	Already in foundation
Maven/Gradle	Must	Already in foundation
Docker	Must	Containerize apps
CI/CD (GitHub Actions)	Must	Automate build/test
Cloud fundamentals	Good	Deployment target
Kubernetes	Awareness	Orchestration
Monitoring/Logging	Awareness	Operations

What Belongs to DevOps/SRE Careers (Not Java Fresher)

- Terraform / Infrastructure as Code
- Advanced Kubernetes (operators, Helm charts, service mesh)
- Prometheus/Grafana deep configuration
- Incident management
- SRE practices (SLOs, error budgets)
- Advanced networking

Recommendation: Learn enough DevOps to deploy your own apps. Do not chase DevOps as a fresher unless you specifically want that career.

PART 10 — JAVA + DSA ROADMAP

Fundamentals (Must Master)

Topic	Fresher Depth	Interview Patterns	Essential?
Big-O	Analyze time/space	All problems	Yes
Arrays	Traversal, rotation, prefix sums	Two pointers, sliding window	Yes
Strings	Manipulation, palindrome, anagrams	Two pointers, hashing	Yes
Hashing	HashMap/HashSet usage	Frequency count, pair sum	Yes
Linked Lists	Traversal, reversal, cycle detection	Fast/slow pointers	Yes
Stack	LIFO, parentheses, next greater	Monotonic stack	Yes
Queue	FIFO, BFS	Level order	Yes

Intermediate (Learn Most)

Topic	Fresher Depth	Patterns	Essential?
Trees	Traversals, height, diameter	DFS, BFS	Yes
BST	Search, insert, validate	In-order traversal	Yes
Heaps	Priority queue, top K	Heap operations	Yes
Recursion	Base case, recurrence	Divide and conquer	Yes
Backtracking	Permutations, combinations	State space tree	Good to have
Binary Search	Sorted arrays, search space	Boundary conditions	Yes
Sorting	Merge, quick, comparison	Custom comparators	Yes
Two Pointers	Sorted arrays, pairs	Sliding window variant	Yes
Sliding Window	Subarrays, substrings	Fixed/variable window	Yes
Prefix Sums	Range queries	Precomputation	Yes

Greedy	Interval scheduling, coins	Local optimal choice	Good to have
--------	----------------------------	----------------------	--------------

Advanced (Awareness for Freshers)

Topic	Fresher Depth	Essential?
Graphs	BFS, DFS, connected components	Basic only
Dynamic Programming	1D DP, knapsack, LCS	Basic patterns
Tries	Insert, search	Not for freshers
Advanced Graphs	Dijkstra, MST, topological sort	Not for freshers

DSA Expectations for MCA Fresher

- **Product companies:** 2–3 LeetCode medium problems in 45 minutes
- **Service companies:** 1–2 easy/medium problems
- **Startups:** Practical coding + basic DSA

Target: Solve 150–200 problems (easy + medium) before interviewing.

PART 11 — JAVA + SQL + DATABASE ROADMAP

SQL Required for Fresher

Topic	Depth
SELECT, WHERE, ORDER BY	Fluent
GROUP BY, HAVING	Fluent
Joins (INNER, LEFT, RIGHT, FULL)	Fluent
Subqueries	Comfortable
CTEs (WITH clause)	Comfortable
Window functions (ROW_NUMBER, RANK)	Basic
Indexes	Conceptual + basic usage
Transactions	Conceptual
ACID	Conceptual
Normalization (1NF–3NF)	Conceptual
Database design	Basic ER modeling
Query optimization	Basic (EXPLAIN)

Advanced Database Knowledge (After Joining)

- Sharding, replication, partitioning

- Advanced indexing strategies
- Query plan analysis
- NoSQL databases
- Database migrations (Flyway, Liquibase)
- Connection pooling (HikariCP)

JDBC, JPA, Hibernate

Technology	Fresher Depth
JDBC	Understand connection, statement, result set. Know what ORM abstracts.
JPA	Entity mapping, relationships, JPQL
Hibernate	Lazy vs eager, caching basics, dirty checking

PART 12 — SYSTEM DESIGN ROADMAP FOR JAVA DEVELOPERS

Beginner (Fresher Interview Level)

Topic	Depth
Client-server architecture	Conceptual
HTTP/REST	Practical
APIs	Design basic REST APIs
Databases	SQL vs NoSQL basics
Caching	What it is, when to use
Basic scalability	Vertical vs horizontal

Intermediate (1–3 Years Experience)

Topic	Depth
Load balancing	Concepts
Database scaling	Read replicas, sharding basics
Redis	Caching patterns
Message queues	Async processing
Microservices	Concepts
CAP theorem	Conceptual
Consistency	Eventual vs strong

Advanced (Senior Level)

Topic	Depth
-------	-------

Distributed systems	Deep
Event-driven architecture	Patterns
High availability	Design
Fault tolerance	Patterns
Distributed caching	Implementation
Distributed databases	Concepts
Observability	Full stack

What Freshers Actually Need

For interviews: Beginner level + ability to design a simple CRUD application (e.g., URL shortener, todo API).

Do not: Pretend to know distributed systems. Be honest about what you know.

PART 13 — COMMON VS SPECIALIZED SKILLS MATRIX

Skill/Chapter	Common Java	Backend	Full Stack	Spring Boot	Microservices	Cloud	QA/Automation	DevOps
Programming Fundamentals	C	C	C	C	C	C	C	C
Core Java (OOP, Collections, Exceptions)	C	C	C	C	C	C	C	C
Java 8+ (Lambdas, Streams)	C	C	C	C	C	C	C	C
Multithreading	C	C	C	C	S	C	C	C
JVM/Memory/GC	C	C	C	C	S	C	C	C
Git	C	C	C	C	C	C	C	C
Maven/Gradle	C	C	C	C	C	C	C	C
Linux	C	C	C	C	C	C	C	S
Unit Testing	C	C	C	C	C	C	S	C
HTTP/REST	C	S	S	S	S	S	S	C
SQL	C	S	S	S	S	S	S	C
JDBC	C	S	S	S	S	S	C	C
Spring Core	C	S	S	S	S	S	A	C
Spring Boot	C	S	S	S	S	S	A	C

Spring MVC	C	S	S	S	S	S	A	C
Spring Data JPA/Hibernate	C	S	S	S	S	S	A	C
Spring Security/JWT	C	S	S	S	S	S	A	C
Docker	C	S	S	S	S	S	C	S
Kubernetes	C	A	A	A	S	S	A	S
Microservices	C	A	A	A	S	S	A	A
Kafka/RabbitMQ	C	A	A	A	S	A	A	A
Cloud (AWS/Azure)	C	A	A	A	S	S	A	S
CI/CD	C	S	S	S	S	S	S	S
Selenium	C	A	A	A	A	A	S	A
REST Assured	C	A	A	A	A	A	S	A
Frontend (HTML/CSS/JS)	C	A	S	A	A	A	A	A
React/Angular	C	A	S	A	A	A	A	A
DSA	C	C	C	C	C	C	C	C
System Design	C	A	A	A	S	S	A	S

Legend: C = Common, S = Specialized, A = Advanced/Optional

PART 14 — SKILL DEPENDENCY MAP

Programming Fundamentals

Core Java (Syntax, OOP, Collections, Exceptions, Generics, I/O)

Java 8+ (Lambdas, Streams, Optional) + Multithreading + JVM Basics

DSA SQL Git Maven/Gradle Linux Testing

JDBC + HTTP + REST + JSON

Spring Core + Spring Boot + Spring MVC + Spring Data JPA

Backend Development (REST APIs, Security, Validation, Testing, Docker)

CHOOSE SPECIALIZATION

Full Stack Microservices Cloud QA/Automation
Frontend Distributed AWS/Azure Selenium
React Kafka Kubernetes REST Assured
TypeScript Resilience CI/CD TestNG

DevOps-oriented
CI/CD + Containers + Cloud

Advanced Backend
Advanced Spring Boot + Performance

PART 15 — PROJECTS

A. Java Backend Developer

Beginner Projects (3)

#	Project	Features	Tech	Skills Demonstrated
1	Student Management REST API	CRUD for students, search by name, pagination	Spring Boot, JPA, H2/MySQL, Maven	REST, JPA, validation, exception handling
2	Todo List API	CRUD todos, mark complete, filter by status	Spring Boot, JPA, PostgreSQL	REST design, status codes, testing
3	Weather API Consumer	Fetch weather from public API, cache results, expose own endpoint	Spring Boot, RestTemplate/WebClient, Caffeine	External API integration, caching

Intermediate Projects (3)

#	Project	Features	Tech	Skills Demonstrated
1	E-Commerce Backend (Mini)	Products, categories, cart, orders, user auth	Spring Boot, JPA, Spring Security, JWT, PostgreSQL	Security, relationships, transactions
2	Blog Platform API	Posts, comments, tags, likes, user roles	Spring Boot, JPA, Spring Security, Flyway	Role-based access, migrations, testing
3	Expense Tracker API	Users, expenses, categories, monthly reports	Spring Boot, JPA, PostgreSQL, Swagger	Reporting queries, API docs

Job-Ready Projects (3)

#	Project	Features	Tech	Skills Demonstrated
---	---------	----------	------	---------------------

1	Job Portal Backend	Companies, jobs, applications, resume upload, search/filter	Spring Boot, JPA, Security, JWT, S3/local storage, Docker	File handling, complex queries, Docker
2	Library Management System	Books, members, loans, fines, reservations, admin dashboard	Spring Boot, JPA, Security, PostgreSQL, Docker, CI/CD	Business logic, scheduling, deployment
3	URL Shortener + Analytics	Shorten URLs, redirect, click analytics, rate limiting	Spring Boot, JPA, Redis (optional), Docker	Caching, analytics, production concerns

GitHub Requirements for Each:

- Meaningful README (problem, features, tech, setup, API docs)
- Proper `.gitignore`
- Clean commit history
- Unit + integration tests
- Postman collection or Swagger
- Docker setup
- Deployed link (if possible)

What Makes It Interview-Worthy:

- Solves a real problem
- Has authentication
- Has proper error handling
- Has tests
- Is deployed
- You can explain design decisions

B. Java Full-Stack Developer

Beginner (3)

#	Project	Frontend	Backend
1	Personal Portfolio	React + Tailwind	Spring Boot (contact form API)
2	Todo App	React + Axios	Spring Boot + JPA
3	Weather Dashboard	React + Chart.js	Spring Boot (API proxy)

Intermediate (3)

#	Project	Frontend	Backend
1	E-Commerce Store	React + Redux (or Context)	Spring Boot + JPA + Security
2	Social Media Feed	React + React Router	Spring Boot + JPA + WebSocket
3	Project Management Tool	React + Drag-and-drop	Spring Boot + JPA + Security

Job-Ready (3)

#	Project	Frontend	Backend
---	---------	----------	---------

1	Learning Management System	React + TypeScript	Spring Boot + JPA + Security + File Upload
2	Food Delivery App	React + Maps	Spring Boot + JPA + WebSocket + Payment simulation
3	Real-Time Chat App	React + Socket.io	Spring Boot + WebSocket + JPA + Security

C. Java Microservices Developer

Beginner (3)

#	Project	Services	Tech
1	Two-Service App	User Service + Order Service	Spring Boot, REST, Eureka
2	API Gateway Demo	Gateway + 2 services	Spring Cloud Gateway
3	Config Server Demo	Config Server + Client	Spring Cloud Config

Intermediate (3)

#	Project	Services	Tech
1	E-Commerce Microservices	Product, Order, User, Gateway	Spring Boot, Eureka, Gateway, JPA
2	Notification System	User Service + Notification Service + Kafka	Kafka, Spring Boot
3	Resilient API	Service with circuit breaker	Resilience4j

Job-Ready (3)

#	Project	Services	Tech
1	Banking Microservices	Account, Transaction, Notification, Gateway	Spring Boot, Kafka, Docker, PostgreSQL
2	Ride-Sharing Backend	Rider, Driver, Trip, Payment	Spring Boot, Kafka, Redis, Docker
3	Monitoring Stack	3 services + Prometheus + Grafana	Micrometer, Prometheus, Grafana

Note: These are for learning. Do not claim production microservices experience as a fresher.

D. Java + Cloud Developer

Beginner (3)

#	Project	Cloud	Tech
1	Deploy Spring Boot to EC2	AWS EC2	Spring Boot, Docker, Nginx
2	Serverless API	AWS Lambda	Spring Boot (or plain Java), API Gateway
3	S3 File Storage	AWS S3	Spring Boot, AWS SDK

Intermediate (3)

#	Project	Cloud	Tech
1	CI/CD Pipeline	GitHub Actions + AWS	Spring Boot, Docker, ECR, EC2
2	RDS-Backed App	AWS RDS	Spring Boot, PostgreSQL
3	CloudWatch Monitoring	AWS CloudWatch	Spring Boot, Micrometer

Job-Ready (3)

#	Project	Cloud	Tech
1	Full Cloud Deployment	AWS (EC2, RDS, S3, IAM)	Spring Boot, Docker, GitHub Actions
2	Containerized App on ECS	AWS ECS	Docker, ECR, ECS
3	Multi-Environment Setup	AWS	Spring Profiles, CI/CD, IaC basics

E. Java QA/Automation

Beginner (3)

#	Project	Tech
1	Manual Test Cases for a Website	Test case design
2	Selenium Login Test	Selenium, JUnit, Maven
3	API Test with REST Assured	REST Assured, JUnit

Intermediate (3)

#	Project	Tech
1	Page Object Model Framework	Selenium, POM, TestNG
2	Data-Driven API Tests	REST Assured, Excel/JSON data
3	CI Test Pipeline	Jenkins/GitHub Actions, Maven

Job-Ready (3)

#	Project	Tech
1	Hybrid Automation Framework	Selenium + REST Assured + TestNG + Maven + ExtentReports
2	E-Commerce Test Suite	Selenium, POM, CI/CD
3	API + UI Integrated Tests	Selenium, REST Assured, TestNG

F. Java DevOps-oriented

Beginner (3)

#	Project	Tech
---	---------	------

1	Dockerize Spring Boot App	Docker, Docker Compose
2	GitHub Actions Build Pipeline	GitHub Actions, Maven
3	Linux Deployment Script	Bash, Linux, SSH

Intermediate (3)

#	Project	Tech
1	CI/CD with Jenkins	Jenkins, Docker, Maven
2	Multi-Container App	Docker Compose, Spring Boot, PostgreSQL
3	Basic Kubernetes Deployment	Minikube, kubectl

Job-Ready (3)

#	Project	Tech
1	Full CI/CD Pipeline	GitHub Actions, Docker, AWS EC2
2	Kubernetes Deployment	K8s, Helm basics
3	Monitoring Stack	Prometheus, Grafana, Spring Boot Actuator

PART 16 — ONE LARGE CAPSTONE PROJECT

Project: “HireHub” — Job Portal Platform

MVP Version (Fresher Level)

Component	Details
Problem Statement	Build a job portal where companies post jobs and candidates apply
Features	User registration/login (candidate/employer), job posting CRUD, job search/filter, apply to job, view applications, resume upload
Tech Stack	Java 17, Spring Boot 3, Spring MVC, Spring Data JPA, Hibernate, Spring Security + JWT, PostgreSQL, Maven, JUnit 5, Mockito, Docker, GitHub Actions
Architecture	Layered monolith: Controller Service Repository DB
Database	Users, Roles, Jobs, Applications, Resumes, Companies
APIs	Auth APIs, Job APIs, Application APIs, User APIs
Testing	Unit tests (service layer), integration tests (repository), controller tests
Deployment	Dockerized, deployed to Render/Railway/AWS EC2
Skills Demonstrated	Full backend stack, security, file handling, testing, Docker, CI/CD
GitHub Requirements	README, API docs (Swagger), Postman collection, Dockerfile, CI workflow

Interview-Worthy Because	Real-world domain, auth, file upload, search, deployment
---------------------------------	--

Advanced Version (After Experience)

Extension	Details
Microservices	Split into User Service, Job Service, Application Service, Notification Service
Kafka	Async notifications (application received, status changed)
Redis	Cache job listings, session management
Kubernetes	Deploy services to K8s
Cloud	AWS ECS/EKS, RDS, S3, CloudWatch
Observability	Prometheus, Grafana, distributed tracing
API Gateway	Spring Cloud Gateway
Service Discovery	Eureka

Fresher Strategy: Build the MVP. Deploy it. Document it. Be able to explain every design decision. Mention the advanced version as “future scope” in your README.

PART 17 — JOB-READY SKILLS

A. Java Backend Developer

Category	Skills
Must Have	Core Java, OOP, Collections, Java 8+, Spring Boot, REST, JPA/Hibernate, SQL, Git, Maven, Unit Testing, Basic Docker
Good to Have	Spring Security, JWT, Swagger, CI/CD, Redis basics
Advanced	Microservices, Kafka, Kubernetes, Cloud
Not Required Initially	Reactive programming, GraphQL, Advanced cloud architecture, Terraform

B. Java Full-Stack Developer

Category	Skills
Must Have	Backend must-haves + HTML, CSS, JavaScript, React/Angular, API integration
Good to Have	TypeScript, Redux/Context, Responsive design
Advanced	SSR, Next.js, Advanced state management, WebSockets
Not Required Initially	Webpack config, Micro-frontends, Advanced animations

C. Spring Boot Developer

Category	Skills
Must Have	Spring Core, Spring Boot, Spring MVC, Spring Data JPA, REST, Security basics, Testing
Good to Have	Actuator, Profiles, Docker, CI/CD
Advanced	Spring Cloud, WebFlux, Batch, Integration
Not Required Initially	Custom auto-config, Advanced performance tuning

D. Java Microservices Developer

Category	Skills
Must Have	Strong backend + Docker + REST + one messaging system
Good to Have	Service discovery, API Gateway, Resilience4j
Advanced	Distributed transactions, Event sourcing, CQRS, Advanced K8s
Not Required Initially	Service mesh, Advanced observability

E. Java + Cloud Developer

Category	Skills
Must Have	Backend + Docker + one cloud provider fundamentals + CI/CD
Good to Have	Kubernetes basics, Cloud databases, IAM
Advanced	Multi-cloud, Terraform, Advanced networking
Not Required Initially	Serverless deep dive, Advanced security

F. Java QA/Automation

Category	Skills
Must Have	Core Java, OOP, Selenium, JUnit/TestNG, REST Assured, Maven, Git
Good to Have	POM design, CI/CD, Postman, SQL
Advanced	Framework architecture, Performance testing, Mobile automation
Not Required Initially	AI-based testing, Complex distributed testing

G. Java DevOps-oriented

Category	Skills
Must Have	Linux, Git, Maven, Docker, CI/CD, Cloud fundamentals
Good to Have	Kubernetes basics, Monitoring
Advanced	Terraform, Advanced K8s, SRE practices
Not Required Initially	Service mesh, Advanced security

General Expectations for MCA Fresher

Area	Expectation
DSA	150–200 problems, easy + medium, patterns
SQL	Joins, GROUP BY, subqueries, basic optimization
Java	Strong OOP, collections, Java 8+, exceptions, basic concurrency
Spring	Build REST APIs with Spring Boot, JPA, Security basics
Projects	2–3 solid projects, deployed, documented
GitHub	Clean profile, READMEs, consistent commits
Resume	1 page, skills + projects + education, no lies
Interview	Explain concepts clearly, code simple problems, discuss projects

PART 18 — INTERVIEW ROADMAP

Track 1: Java Developer (Generic)

Topic	Depth
Core Java	Strong
OOP	Strong
Collections	Strong
Multithreading	Conceptual
Java 8+	Strong
JVM Basics	Conceptual
DSA	Easy–Medium
SQL	Moderate
DBMS	Conceptual
OS Basics	Conceptual
Networks	Conceptual
Spring	Basic
Projects	Strong
Behavioral	STAR method

Track 2: Java Backend Developer

Topic	Depth
-------	-------

Core Java	Strong
Collections	Strong
Multithreading	Moderate
Java 8+	Strong
JVM	Moderate
DSA	Medium
SQL	Strong
Spring Boot	Strong
REST	Strong
JPA/Hibernate	Strong
Security	Moderate
System Design	Basic
Projects	Very Strong

Track 3: Spring Boot Developer

Topic	Depth
Spring Core	Strong
Spring Boot	Very Strong
Spring MVC	Strong
Spring Data JPA	Strong
Security	Moderate
Testing	Strong
REST	Strong
Docker	Basic
Projects	Very Strong

Track 4: Full-Stack Java Developer

Topic	Depth
Backend Track	Strong
HTML/CSS	Moderate
JavaScript	Strong
React/Angular	Strong

API Integration	Strong
Projects	Very Strong

Track 5: Java Microservices Developer

Topic	Depth
Backend Track	Very Strong
Microservices Concepts	Strong
Spring Cloud	Moderate
Docker	Strong
Kafka	Moderate
Kubernetes	Basic
System Design	Moderate
Projects	Strong

Track 6: Java QA/Automation Engineer

Topic	Depth
Core Java	Strong
OOP	Strong
Selenium	Strong
TestNG/JUnit	Strong
REST Assured	Strong
API Testing	Strong
SQL	Moderate
CI/CD	Moderate
Framework Design	Moderate
Projects	Strong

Expected Depth for MCA Fresher

- **Core Java:** Can explain and code
 - **Spring Boot:** Can build a REST API from scratch
 - **SQL:** Can write joins and group by queries
 - **DSA:** Can solve easy and some medium problems
 - **Projects:** Can explain architecture, challenges, decisions
 - **Behavioral:** Can talk about teamwork, learning, challenges
-

PART 19 — TIME-BASED ROADMAP

Assumptions

- MCA completed
 - Basic programming knowledge
 - No professional experience
 - Goal: first entry-level job
 - 4–6 hours/day dedicated learning
-

3-Month Roadmap (Aggressive)

Month	Focus	Outcome
Month 1	Core Java + OOP + Collections + Java 8+	Strong Java foundation
Month 2	SQL + JDBC + HTTP + Spring + Spring Boot	Build first REST API
Month 3	Spring Boot project + DSA + Interview prep	Job-ready for service companies

Note: Aggressive. Suitable if you can study 6+ hours/day.

6-Month Roadmap (Recommended)

Month	Focus	Outcome
Month 1	Programming + Core Java + OOP	Java fundamentals
Month 2	Collections + Exceptions + Generics + I/O + Java 8+	Core Java complete
Month 3	DSA (arrays, strings, hashing, linked lists, stacks, queues) + SQL	DSA + SQL basics
Month 4	HTTP + REST + JDBC + Spring Core + Spring Boot	Backend foundation
Month 5	Spring Data JPA + Security + Testing + Docker + Project 1	First backend project
Month 6	Project 2 + DSA (trees, graphs) + Interview prep	Job applications

9-Month Roadmap (Comfortable)

Month	Focus
Month 1	Programming + Core Java
Month 2	OOP + Collections + Exceptions
Month 3	Java 8+ + I/O + Multithreading basics
Month 4	DSA (fundamentals + intermediate)
Month 5	SQL + DBMS + JDBC
Month 6	HTTP + REST + Spring + Spring Boot
Month 7	Spring Data JPA + Security + Testing

Month 8	Project 1 + Project 2 + Docker
Month 9	DSA revision + Interview prep + Job applications

12-Month Roadmap (Thorough)

Month	Focus
Months 1–2	Core Java + OOP + Collections + Java 8+
Month 3	Multithreading + JVM + I/O
Months 4–5	DSA (all fundamentals + intermediate)
Month 6	SQL + DBMS + JDBC
Month 7	HTTP + REST + Spring + Spring Boot
Month 8	Spring Data JPA + Security + Testing
Month 9	Project 1 + Docker + CI/CD
Month 10	Project 2 + System Design basics
Month 11	DSA advanced + Interview prep
Month 12	Job applications + Mock interviews

PART 20 — RECOMMENDED LEARNING STRATEGY

Improved Sequence

Phase 1: Programming + Core Java

(Syntax, OOP, Collections, Exceptions, Generics, I/O, Java 8+)

Duration: 6–8 weeks

Phase 2: DSA + SQL + Git + Linux + Maven

(Build problem-solving and tooling foundation)

Duration: 6–8 weeks

Phase 3: JDBC + HTTP + REST + JSON

(Understand how backend communicates)

Duration: 2–3 weeks

Phase 4: Spring + Spring Boot + Spring MVC + Spring Data JPA

(Build REST APIs)

Duration: 4–6 weeks

Phase 5: Spring Security + JWT + Validation + Exception Handling + Testing

(Production-ready backend)

Duration: 3–4 weeks

Phase 6: Backend Projects (2–3 solid projects)

(Apply everything, build portfolio)

Duration: 6–8 weeks

Phase 7: Choose Specialization

Full Stack: Learn React/Angular

Microservices: Learn Spring Cloud, Kafka, Docker

Cloud: Learn AWS/Azure, Docker, K8s

QA: Learn Selenium, REST Assured

DevOps: Learn CI/CD, K8s, Cloud

Duration: 4–8 weeks

Phase 8: Advanced Specialization Projects

(Build 1–2 specialization-specific projects)

Duration: 4–6 weeks

Phase 9: Interview Preparation

(DSA revision, mock interviews, behavioral prep)

Duration: 4–6 weeks (ongoing)

Key Principles

1. **Code every day.** Even 1 hour of coding beats 3 hours of watching.
2. **Build projects, not just tutorials.** Tutorials are for learning; projects are for proving.
3. **Learn just enough frontend** if going full-stack. Don't become a frontend expert.
4. **Don't learn microservices before backend.** It will confuse you.
5. **Don't chase certifications.** Build projects instead.
6. **Apply for jobs before you feel "ready."** You'll learn more from interviews.
7. **GitHub is your portfolio.** Keep it clean.
8. **DSA is a marathon, not a sprint.** Practice consistently.

PART 21 — FINAL COMPARISON

Factor	Java Backend	Java Full Stack	Spring Boot	Microservices	Java + Cloud	Java QA/Automation	Java DevOps-oriented
Core Java Requirement	High	High	High	High	High	Moderate-High	Moderate-High
DSA Requirement	Moderate	Moderate	Moderate	Moderate	Moderate	Low-Moderate	Low-Moderate
SQL Requirement	High	High	High	High	High	Moderate	Moderate
Spring Requirement	High	High	Very High	Very High	High	Low	Moderate
Frontend Requirement	None	High	None	None	None	None	None

Cloud Requirement	Low	Low	Low	High	Very High	Low	High
DevOps Requirement	Low	Low	Low	Moderate	High	Moderate	Very High
Testing Requirement	Moderate	Moderate	High	High	Moderate	Very High	Moderate
System Design Requirement	Basic	Basic	Basic	High	High	Low	Moderate
Main Technologies	Spring Boot, JPA, SQL	Spring Boot + React	Spring Boot, JPA	Spring Cloud, Kafka, Docker, K8s	AWS/Azure, Docker, K8s	Selenium, REST Assured	Docker, K8s, CI/CD, Cloud
Entry-Level Responsibilities	Build APIs, fix bugs, write tests	Build UI + APIs	Build APIs, configure Spring	(Not entry-level)	Deploy apps, basic cloud	Write automated tests	Automate builds, deployments

FINAL HIERARCHY

COMMON JAVA FOUNDATION

Programming Fundamentals Core Java OOP Collections
 Exceptions Generics I/O Java 8+ Multithreading
 JVM Basics Git Maven Linux Testing HTTP

CORE JAVA + DSA + SQL + GIT + LINUX + MAVEN

JDBC + HTTP + REST + JSON

SPRING + SPRING BOOT + SPRING MVC + JPA

BACKEND DEVELOPMENT (REST APIs,
 Security, Validation, Testing, Docker)

CHOOSE SPECIALIZATION

Full Stack	Microservices	Cloud	QA/Automation	DevOps
React	Kafka	AWS/Azure	Selenium	CI/CD
TypeScript	Resilience	K8s	REST Assured	K8s
Frontend	Docker	CI/CD	TestNG	Cloud

ADVANCED BACKEND

Advanced Spring Boot + Performance

KEY DIFFERENCES EXPLAINED

Comparison	Explanation
Java Developer vs Java Backend Developer	Java Developer is generic (could be backend, desktop, etc.). Backend Developer specifically builds server-side applications. In modern industry, they are often the same.
Java Backend vs Spring Boot Developer	Spring Boot Developer is a backend developer who specializes in Spring Boot. Backend is broader (could use other frameworks).
Backend vs Full Stack	Backend = server-side only. Full Stack = backend + frontend.
Spring Boot vs Microservices	Spring Boot is a framework for building applications. Microservices is an architectural style. You can build microservices with Spring Boot, but Spring Boot $\neq$ microservices.
Java Developer vs Java + Cloud Developer	Cloud Developer adds cloud platform skills (AWS/Azure) to Java development.
Java Developer vs Java QA/Automation	Developer builds features. QA/Automation builds test automation. Different career trajectories.
Java Developer vs DevOps Engineer	Developer builds applications. DevOps engineer builds and maintains the infrastructure and pipelines that deploy applications.

FINAL WORDS OF ADVICE

1. **You don't need to learn everything.** Master the common foundation, then choose one specialization.
2. **Backend is the safest entry point** for MCA freshers. Full-stack is a close second.
3. **Microservices, Cloud, and DevOps are not entry-level specializations.** Learn them after your first job.
4. **Projects matter more than certificates.** Build 2–3 solid, deployed projects.
5. **DSA and SQL are non-negotiable** for most entry-level interviews.
6. **Start applying before you feel ready.** Interview experience is invaluable.
7. **Be honest in interviews.** Say "I haven't worked with that, but I understand the concept and can learn."
8. **Your first job is a learning opportunity.** Don't optimize for salary; optimize for growth.

Good luck. Now go build something.